

SIMATS ENGINEERING ASSESSMENT TOOL₃

COURSE NAME: MLA0303
COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)

Assessment Tool Weightage: 50%

Dr Dumpala Prasanth

QUESTIONS: 10

Total Marks:50

Annexure A

Constraint - Based Problem-Solving

Duration: 2Hrs

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.
(10 Marks)
2. Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.
(10 Marks)
3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.
(10 Marks)
4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.
(10 Marks)
5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of

deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

(10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and ϵ -greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

(10 Marks)

Code of Conduct:

I Charamdeep.NC / 192472196 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

CO1AT3

1. Python based Reinforcement learning using OpenAI gym/Grid World

Problem Understanding

Develop a Reinforcement Learning (RL) agent that navigates a grid world to reach the goal while avoiding obstacles. The objective is to maximize the total reward.

Constraint Analysis

- Agent cannot move outside the grid.
- Obstacles cannot be crossed.
- Goal state ends the episode.
- Each move has a small penalty to encourage shorter paths.

Solution Development

State Space

Each grid cell represents one state.

Ex:

S	.	.	X
.	X	.	.
.	.	.	X
.	.	.	G

• S = Start

• G = Goal

• X = Obstacle

States = 16

Action space

- up
- Down
- Left
- Right

Reward Function

- Goal = +10
- Obstacle = -10
- Every step = -1

Python Code

```
import gym
env = gym.make("Frozenlake-V1", is_slippery = False)
State = env.reset()
done = False
while not done:
    action = env.action_space.sample()
    info = env.step(action)
    Print(State, reward).
```

Justification

The reward function encourages the shortest safe path while Penalties prevent collisions with obstacles.

2. ϵ -Greedy Multi-Armed Bandit Algorithm

Problem Understanding

Implement an ϵ -greedy algorithm to maximize reward within a budget of 500 trials.

Constraint Analysis

- Total trials = 500
- Exploration should be balanced
- Higher reward is preferred

Solution Development

Code:

```
import random
```

$\epsilon = 0.1$

arms = [0, 0, 0]

counts = [0, 0, 0]

for i in range(500):

if random.random() < ϵ :

arm = random.randint(0, 2)

else:

arm = arms.index(max(arms))

reward = random.randint(0, 1)

counts[arm] += 1

arms[arm] += (reward - arms[arm]) / counts[arm]

Print(arms)

Comparison

ϵ value	Exploration	Exploitation	Performance
0.1	low	high	Highest reward
0.3	Medium	Balanced	Good
0.5	High	Low	Lower reward

Applications

- Exploration
- Exploitation
- Multi-Armed Bandit
- ϵ -Greedy Policy

Justification

$\epsilon = 0.1$ Provides better long term reward because it focuses more on exploiting the best-performing arm.

3. Markov Decision Process (MDP) for Autonomous Robot.

Problem Understanding

Design an MDP where a robot reaches its destination while minimizing energy consumption.

Constraint Analysis

- Battery is limited.
- Robot must avoid unnecessary movements.
- Goal should be reached using minimum energy.

Solution Development

States

- Start
- Intermediate cells
- Goal

Actions

- up
- Down
- Left
- Right.

Transition Probability

Correct action = 0.9

wrong direction = 0.1

Reward Function

Goal : +100

Every movement : -1

High energy usage : -5

Energy Constraint

Maximum Energy : 50 units

Robot fails if energy becomes zero before reaching destination.

Justification
Negative rewards reduce unnecessary movement and help conserve battery energy.

4. Value Iteration using Python (TensorFlow/Keras)

Problem understanding.

Determine the optimal Policy in a constrained grid world using Value Iteration

Constraint Analysis.

- Restricted states cannot be entered.
- Agent should maximize future rewards.
- Goal must be reached efficiently.

Solution Development

Bellman Equation

$$V(s) = \max_a \sum P(s'|s, a) [R + \gamma V(s')]$$

Code

```
import numpy as np
gamma = 0.9
V = np.zeros((4,4))
reward = np.zeros((4,4))
reward[3][2] = 10

for k in range(100):
    for i in range(4):
        for j in range(4):
            gamma * np.max(V)
```

Justification

Bellman updates repeatedly improve Value estimates until convergence, producing the optimal Policy.

5. RL Framework for warehouse Robot

Problem understanding

A warehouse robot has only 30 minutes of battery life and must complete the maximum number of deliveries.

Constraint Analysis

- Battery Capacity = 30 minutes.
- Robot should return before battery is exhausted.
- Deliveries should be maximized.

Solution development

States

- Current location
- Battery level
- Delivery status

Actions

- Move
- Pick Package
- Deliver Package.

Reward design

Successful delivery:
+20

Battery saving:
+5

Battery exhausted:
-50

Energy management:
-1

Justification

The reward structure encourages efficient deliveries while ensuring the battery limit is never exceeded, maximizing completed deliveries safely.